

السلاسل النصية والتعامل معها

«السلسلة النصية» (string) هي مجموعة من المحارف (أي الأحرف والأرقام والرموز) التي إما أن تكون قيمة ثابتة أو قيمة لمتغير. وهذه السلاسل النصية مُشكَّلة من محارف يونيكود (Unicode) والتي لا يمكن تغيير مدلولها. ولأنَّ النص هو شكلٌ شائعٌ من أشكال البيانات الذي نستعمله يوميًا، لذا فإنَّ السلاسل النصية مهمة جدًا وتُمثِّلُ بُنيةً أساسيةً في البرمجة. سيستعرض هذا الفصل كيفية إنشاء وطباعة السلاسل النصية، وكيفية جمعها مع بعضها وتكرارها، وآلية تخزين السلاسل النصية في متغيرات.

1. إنشاء وطباعة السلاسل النصية

تتواجد السلاسل النصية إما داخل علامات اقتباس فردية ' أو علامات اقتباس مزدوجة "، لذا لإنشاء سلسلة نصية، كل ما علينا فعله هو وضع مجموعة من المحارف بين أحد نوعي علامات الاقتباس السابقين:

'هذه سلسلة نصية ضمن علامتي اقتباس مفردتين'
"هذه سلسلة نصية ضمن علامتي اقتباس مزدوجتين"

يمكنك الاختيار بين النوعين السابقين، لكن أياً كان اختيارك، فعليك أن تحافظ على استخدامك له في كامل برنامجك. يمكنك طباعة السلاسل النصية إلى الشاشة باستدعاء الدالة `print()` بكل بساطة:

```
print("Let's print out this string.")
Let's print out this string.
```

بعد أن فهمت كيفية تهيئة السلاسل النصية في بايثون، لنلقِ نظرةً الآن إلى كيفية التعامل مع السلاسل النصية في برامجك وتعديلها.

2. آلية فهرسة السلاسل النصية

وكما في نوع البيانات list الذي فيه عناصر مرتبطة بأرقام، فإن كل محرف في السلسلة النصية يرتبط بفهرس معيّن، بدءًا من الفهرس 0. فللسلسلة النصية !Sammy Shark ستكون الفهارس والمحارف المرتبطة بها كالآتي:

s a m m y s h a r k !											
0	1	2	3	4	5	6	7	8	9	10	11

وكما لاحظت، سيرتبط المحرف s بالفهرس 0، وستنتهي السلسلة النصية بالفهرس 11 مع الرمز !. لاحظ أيضًا أن الفراغ بين كلمتي Sammy و Shark له فهرس خاص به، وفي مثالنا سيكون له الفهرس 5. علامة التعجب ! لها فهرس خاص بها أيضًا، وأيّة رموز أو علامات ترقيم أخرى مثل *\$#&. ; ? سترتبط بفهرس مخصص لها. حقيقة أن المحارف في بايثون لها فهرس خاص بها ستعني أن بإمكاننا الوصول إلى السلاسل النصية وتعديلها كما نفعل مع أنواع البيانات المتسلسلة الأخرى.

1. الوصول إلى المحارف بفهارس موجبة

يمكننا الحصول على محرف من سلسلة نصية بالإشارة إليه عبر فهرسه. يمكننا فعل ذلك بوضع رقم الفهرس بين قوسين معقوفين []. سنعرّف في المثال الآتي سلسلة نصية ونطبع المحرف المرتبط بالفهرس المذكور بين قوسين:

```
ss = "Sammy Shark!"
print(ss[4])
```

النتائج:

y

عندما نُشير إلى فهرس معيّن في سلسلة نصية، فستُعيد بايثون المحرف الموجود في ذلك الموضع، ولما كان المحرف y موجودًا في الفهرس الرابع في السلسلة النصية "Sammy Shark!" فعندما طبعنا ss[4] فظهر الحرف y.

خلاصة ما سبق، أرقام الفهارس ستسمح لنا بالوصول إلى محارف معينة ضمن سلسلة نصية.

ب. الوصول إلى المحارف بفهارس سالبة

إذا كانت لديك سلسلة طويلة وأردنا تحديد أحد محارفها لكن انطلاقًا من نهايتها، فعندئذٍ نستطيع استخدام الأرقام السالبة للفهارس، بدءًا من الفهرس -1. لو أردنا أن نستخدم الفهارس السالبة مع السلسلة النصية "Sammy Shark!"، فستبدو كما يلي:

s	a	m	m	y		s	h	a	r	k	!
-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

يمكننا أن نطبع المحرف r في السلسلة السابقة في حال استخدمنا الفهارس السالبة بالإشارة إلى المحرف الموجود في الفهرس -3 كما يلي:

```
print(ss[-3])
```

وجدنا أنّه يمكننا الاستفادة من الفهارس السالبة لو أردنا الوصول إلى محرف في آخر سلسلة نصية طويلة.

3. تقسيم السلاسل النصية

يمكننا أن نحصل على مجال من المحارف من سلسلة نصية، فلنقل مثلاً أننا نريد أن نطبع الكلمة Shark فقط، يمكننا فعل ذلك بإنشائنا «لقسم» من السلسلة النصية، والذي هو سلسلة من المحارف الموجودة ضمن السلسلة الأصلية. فالأقسام تسمح لنا بالوصول إلى عدّة محارف دفعةً واحدة باستعمال مجال من أرقام الفهارس مفصولة فيما بينها بنقطتين رأسيّتين [x:y]:

```
print(ss[6:11])
```

النتائج:

Shark

عند إنشائنا لقسم مثل [6:11] فسيُمثّل أوّل رقم مكان بدء القسم (متضمناً المحرف الموجود عند ذاك الفهرس)، والرقم الثاني هو مكان نهاية القسم (دون تضمين ذاك المحرف)، وهذا هو السبب وراء استخدامنا لرقم فهرس يقع بعد نهاية القسم الذي نريد اقتطاعه في المثال السابق. نحن نُنشئ «سلسلة نصية فرعية» (substring) عندما نُقسّم السلاسل النصية، والتي هي سلسلة موجودة ضمن سلسلة أخرى. وعندما نستخدم التعبير ss[6:11] فنحن نستدعي السلسلة النصية Shark التي تتواجد ضمن السلسلة النصية Sammy Shark. إذا أردنا تضمين نهاية السلسلة (أو بدايتها) في القسم الذي سننشئه، فيمكن ألا نضع أحد أرقام الفهارس في string[n:n]. فمثلاً، نستطيع أن نطبع أوّل كلمة من السلسلة ss - أي Sammy - بكتابة ما يلي:

```
print(ss[:5])
```

فعلنا ذلك بحذف رقم الفهرس قبل النقطتين الرأسيتين، ووضعا رقم فهرس النهاية فقط، الذي يُشير إلى مكان إيقاف اقتطاع السلسلة النصية الفرعية. لطباعة منتصف السلسلة النصية إلى آخرها، فسنضع فهرس البداية فقط قبل النقطتين الرأسيتين، كما يلي:

```
print(ss[7:])
```

النتاج:

```
hark!
```

بكتابة فهرس البداية فقط قبل النقطتين الرأسيتين وترك تحديد الفهرس الثاني، فإنَّ السلسلة الفرعية ستبدأ من الفهرس الأول إلى نهاية السلسلة النصية كلها. يمكنك أيضًا استخدام الفهارس السالبة في تقسيم سلسلة نصية، فكما ذكرنا سابقًا، تبدأ أرقام الفهارس السالبة من الرقم 1-، ويستمر العد إلى أن نصل إلى بداية السلسلة النصية. وعند استخدام الفهارس السالبة فسنبداً من الرقم الأصغر لأنَّه يقع أولاً في السلسلة. لنستخدم فهرسين ذوي رقمين سالبين لاقتطاع جزء من السلسلة النصية ss:

```
print(ss[-4:-1])
```

النتاج:

```
ark
```

السلسلة النصية "ark" مأخوذة من السلسلة النصية "Sammy Shark!" لأنَّ الحرف a يقع في الموضع 4- والحرف k يقع قبل الفهرس 1- مباشرةً.

يمكن تحديد الخطوة عند تقسيم السلاسل النصية وذلك بتمرير معامل ثالث إضافةً إلى فهرسي البداية والنهاية، وهو الخطوة، التي تُشير إلى عدد المحارف التي يجب تجاوزها بعد

الحصول على المحرف من السلسلة النصية. لم تُحدّد إلى الآن الخطوة في أمثلتنا، إلا أنّ قيمته الافتراضية هي 1، لذا سنحصل على كل محرف يقع بين الفهرسين. لننظر مرةً أخرى إلى المثال السابق الذي يطبع السلسلة النصية الفرعية "Shark":

```
print(ss[6:11]) # Shark
```

سنحصل على نفس النتائج بتضمين معامل ثالث هو الخطوة وقيمه 1:

```
print(ss[6:11:1]) # Shark
```

إذاً، إذا كانت الخطوة 1 فهذا يعني أنّ بايثون ستُضمّن جميع المحارف بين فهرسين، وإذا حذفّت الخطوة فستعدها بايثون مساويةً للواحد. أما لو زدنا الخطوة، فسنرى أنّ بعض المحارف ستهمل:

```
print(ss[0:12:2]) # SmySak
```

تحديد الخطوة بقيمة 2 كما في `ss[0:12:2]` سيؤدي إلى تجاوز حرف بين كل حرفين، ألقي نظرةً على المحارف المكتوبة بخط عريض:

```
Sammy Shark!
```

لاحظ أنّ الفراغ الموجود في الفهرس 5 قد أهمل أيضًا عندما كانت الخطوة 2. إذا وضعنا قيمةً أكبر للخطوة، فسنحصل على سلسلة نصيةً فرعيةً أصغر بكثير:

```
print(ss[0:12:4]) # Sya
```

حذف الفهرسين وترك النقطتين الرأسيتين سيؤدي إلى إبقاء كامل السلسلة ضمن المجال، لكن إضافة معامل ثالث وهو الخطوة سيحدّد عدد المحارف التي سيتم تخطيها. إضافةً إلى ذلك، يمكنك تحديد رقم سالب كخطوة، مما يمكّنك من كتابة السلسلة النصية بترتيب معكوس إذا

استعملت القيمة 1-:

```
print(ss[::-1])
```

الناتج:

```
!krahS ymmaS
```

لنجرّب ذلك مرةً أخرى لكن إذا كانت الخطوة 2-:

```
print(ss[::-2])
```

الناتج:

```
!rh ma
```

في المثال السابق (`ss[::-2]`)، سنتعامل مع كامل السلسلة النصية لعدم وجود أرقام لفهارس البداية والنهاية، وسيتم قلب اتجاه السلسلة النصية لاستخدامنا لخطوة سالبة. بالإضافة إلى أنّ الخطوة 2- ستؤدي إلى تخطي حرف بين كل حرفين بترتيب معكوس:

```
!krahS[whitespace]ymmaS
```

سيُطَبَع الفراغ في المثال السابق.

ما رأيناه هو أنّ تحديد المعامل الثالث عند تقسيم السلاسل النصية سيؤدي إلى تحديد الخطوة التي تُمثّل عدد المحارف التي سيتم تخطيها عند الحصول على السلسلة الفرعية من السلسلة الأصلية.

4. جمع السلاسل النصية

عملية الجمع (concatenation) تعني إضافة سلسلتين نصيتين إلى بعضهما بعضًا لإنشاء سلسلة نصية جديدة. نستخدم العامل + لجمع السلاسل النصية؛ أبقِ في ذهنك أنَّ العامل + يعني عملية الجمع عند التعامل مع الأعداد، أما عندما نستخدمه مع السلاسل النصية فيعني إضافتها إلى بعضها. لنجمع السلسلتين النصيتين "Sammy" و "Shark" مع بعضها ثم نطبعهما باستخدام الدالة `print()`:

```
print("Sammy" + "Shark")
# SammyShark
```

إذا أردتَ وضع فراغ بين السلسلتين النصيتين، فيمكنك بكل بساطة وضعه عند نهاية السلسلة النصية الأولى، أي بعد الكلمة "Sammy":

```
print("Sammy " + "Shark")
# Sammy Shark
```

لكن احرص على عدم استعمال العامل + بين نوعين مختلفين من البيانات، فلن تتمكن من جمع السلاسل النصية والأرقام مع بعضها، فلو حاولنا مثلًا أن نكتب:

```
print("Sammy" + 27)
```

فسنحصل على رسالة الخطأ الآتية:

```
TypeError: Can't convert 'int' object to str implicitly
```

أما إذا أردنا أن نُنشئ السلسلة النصية "Sammy27" فعلينا حينها وضع الرقم 27 بين علامتي اقتباس ("27") مما يجعله سلسلة نصية وليست عددًا صحيحًا. سنستفيد من تحويل الأعداد إلى سلاسل نصية عندما نتعامل مع أرقام الهواتف على سبيل المثال، لأننا لن نحتاج إلى

إجراء عملية حسابية على رمز الدولة ورمز المنطقة في أرقام الهواتف، إلا أننا نريدهما أن يظهرًا متتابعين. عندما نجمع سلسلتين نصيتين أو أكثر فنحن نُنشئ سلسلة نصية جديدة التي يمكننا استخدامها في برنامجنا.

5. تكرار السلاسل النصية

هناك أوقات نحتاج فيها إلى استخدام بايثون لأتمتة المهام، وإحدى الأمور التي يمكننا أتمتتها هي تكرار سلسلة نصية لعدة مرات. إذ نستطيع فعل ذلك عبر العامل `*`، وكما هو الأمر مع العامل `+` فإن العامل `*` له استخدام مختلف عندما نتعامل مع أرقام، حيث يُمثل عملية الضرب. أمّا عندما نستخدمه بين سلسلة نصية ورقم فإن العامل `*` هو معامل التكرار، فوظيفته هي تكرار سلسلة نصية لأي عدد مرات تشاء. لنحاول طباعة السلسلة النصية "Sammy" تسع مرات دون تكرارها يدويًا، وذلك عبر العامل `*`:

```
print("Sammy" * 9)
```

المخرجات:

```
SammySammySammySammySammySammySammySammySammy
```

يمكننا بهذه الطريقة تكرار السلسلة النصية لأي عدد نشاء من المرات.

6. تخزين السلاسل النصية في متغيرات

وجدنا من فصل المتغيرات أنّ المتغيرات هي «رموز» يمكننا استعمالها لتخزين البيانات في برنامج. أي يمكنك تخيل المتغيرات على أنها صندوق فارغ يمكنك ملؤه بالبيانات أو القيم. السلاسل النصية هي نوع من أنواع البيانات، لذا يمكننا استعمالها لملء المتغيرات. التصريح عن

متغيرات تحوي سلاسل نصية سيُسَهِّل علينا التعامل معها في برامجنا. لتخزين سلسلة نصية داخل متغير، فكل ما علينا فعله هو إسنادها إليه. سنُصَرِّح في المثال الآتي عن المتغير `my_str`:

```
my_str = "Sammy likes declaring strings."
```

أصبح المتغير `my_str` الآن مُشيرًا إلى سلسلة نصية، والتي أمسى بمقدورنا طباعتها كما يلي:

```
print(my_str)
```

وسنحصل على الناتج الآتي:

```
Sammy likes declaring strings.
```

استخدام المتغيرات لاحتواء قيم السلاسل النصية سيساعدنا في الاستغناء عن إعادة كتابة السلسلة النصية في كل مرة نحتاج استخدامها، مما يُبَسِّط تعاملنا معها وإجراءنا للعمليات عليها في برامجنا.

7. دوال السلاسل النصية

لدى بايثون عدَّة دوال مبنية فيها للتعامل مع [السلاسل النصية](#). تسمح هذه الدوال لنا بتعديل وإجراء عمليات على السلاسل النصية بسهولة. يمكنك أن تتخيل الدوال على أنَّها «أفعال» يمكننا تنفيذها على عناصر موجودة في الشيفرة. الدوال المبنية في اللغة هي الدوال المُعرَّفة داخل لغة بايثون وهي جاهزة مباشرةً للاستخدام. سنشرح في هذا القسم مختلف الدوال التي نستطيع استخدامها للتعامل مع السلاسل النصية في بايثون 3.

١. جعل السلاسل النصية بأحرف كبيرة أو صغيرة

الدالتان `str.lower()` و `str.upper()` ستُعيدان السلسلة النصية بعد تحويل حالة جميع أحرفها الأصليّة إلى الأحرف الكبيرة أو الصغيرة (على التوالي وبالترتيب). ولعدم قدرتنا على تعديل السلسلة النصية بعد إنشائها، فسُعاد سلسلّة نصيّة جديدة. لن تُعدّل أيّة محارف غير لاتينية في السلسلة النصية الأصليّة وستبقى على حالها. لنحوّل السلسلة النصية `Sammy Shark` إلى أحرفٍ كبيرة:

```
ss = "Sammy Shark"
print(ss.upper())
```

الناتج:

```
SAMMY SHARK
```

لنحوّلها الآن إلى أحرفٍ صغيرة:

```
print(ss.lower())
```

وسينتج:

```
sammy shark
```

سُسهّل الدالتان `str.lower()` و `str.upper()` عملية التحقق من مساواة سلسلتين نصيتين لبعضهما أو لموازنتهما وذلك عبر توحيد حالة الأحرف. فلو كتب المستخدم اسمه بأحرف صغيرة فسَنستطيع أن نتأكد إن كان مُسجَّلًا في قاعدة البيانات بموازنته بعد تحويل حالة أحرفه.

ب. الدوال المنطقية

تتوفر في بايثون عدّة دوال تتحقق من القيم المنطقية (Boolean). هذه الدوال مفيدة عند

إنشائنا للنماذج التي يجب على المستخدمين ملأها؛ فمثلاً، إذا سألنا المستخدم عن الرمز البريدي وأردنا أن نقبل السلاسل النصية التي تحتوي أرقامًا فقط، أو عندما نسأله عن اسمه فسنقبل سلسلة نصية تحوي حروفًا فقط. هنالك عددٌ من الدوال التي تُعيد قيمًا منطقية:

- `str.isalnum()`: تتحقّق إذا احتوت السلسلة النصية على أرقام وأحرف فقط (دون رموز)، أي تعيد `true` إن تحقّق ذلك.
- `str.isalpha()`: تتحقّق إذا احتوت السلسلة النصية على أحرف فقط (دون أرقام أو رموز).
- `str.islower()`: تتحقّق إذا كانت جميع أحرف السلسلة النصية صغيرة.
- `str.isnumeric()`: تتحقّق إذا احتوت السلسلة النصية على أرقام فقط.
- `str.isspace()`: تتحقّق إذا لم تحتوي السلسلة النصية إلا على الفراغات.
- `str.istitle()`: تتحقّق إذا كانت حالة أحرف السلسلة النصية كما لو أنّها عنوان باللغة الإنجليزية (أي أنّ أوّل حرف من كل كلمة كبير، والبقية صغيرة).
- `str.isupper()`: تتحقّق إذا كانت جميع أحرف السلسلة النصية كبيرة.

لنجرّب استعمال بعضها عمليًا:

```
number = "5"
letters = "abcdef"

print(number.isnumeric())
print(letters.isnumeric())
```

الناتج:

True
False

استخدام الدالة `str.isnumeric()` على السلسلة النصية 5 سيُعيد القيمة `True`، بينما استخدام نفس الدالة على السلسلة النصية `abcdef` سيُعيد `False`. وبشكلٍ مماثل، يمكننا معرفة إن كانت حالة الأحرف في سلسلة نصية كما لو أنَّها عنوان، أو أنها كبيرة أو صغيرة (هذه العملية تنطبق على اللغات المكتوبة بالأحرف اللاتينية). لُنُنْشِ بدايةً بعض السلاسل النصية:

```
movie = "2001: A SAMMY ODYSSEY"
book = "A Thousand Splendid Sharks"
poem = "sammy lived in a pretty how town"
```

لنجرَّب الدوال المنطقية لمعرفة الناتج (سنعرض كل دالتين ونتجهما تحتها):

```
print(movie.islower())
print(movie.isupper())

# False
# True

print(book.istitle())
print(book.isupper())

# True
# False

print(poem.istitle())
print(poem.islower())

# False
# True
```

ستساعدنا معرفة إن كانت أحرف السلسلة النصية بحالة صغيرة أو كبيرة أو كأنها عنوان في تصنيف البيانات تصنيفًا سليمًا، وتوفّر لنا الفرصة لتوحيد طريقة تخزين البيانات بالتحقق من حالة أحرفها ثم تعديلها وفقًا لذلك. الدوال المنطقية التي تعمل على السلاسل النصية مفيدة أيضًا عندما نريد التحقق إن حققت مدخلات المستخدم شروطًا معينة.

ج. الدوال `join()` و `split()` و `replace()`

توفّر الدوال `str.join()` و `str.split()` و `str.replace()` إمكانيات إضافية لتعديل السلاسل النصية في بايثون. الدالة `str.join()` تجمع سلسلتين نصيتين مع بعضهما، لكنّها تفعل ذلك بتمرير إحداها إلى الأخرى. لننشئ سلسلة نصية:

```
balloon = "Sammy has a balloon."
```

لنستخدم الآن الدالة `str.join()` لإضافة فراغات إلى تلك السلسلة النصية كالآتي:

```
" ".join(balloon)
```

إذا طبعنا الناتج:

```
print(" ".join(balloon))
```

فسنجد أنّ السلسلة النصية الجديدة هي السلسلة الأولى لكن بين كل حرفين فراغ:

```
S a m m y   h a s   a   b a l l o o n .
```

يمكننا أيضًا استخدام الدالة `str.join()` لإنشاء مقلوب سلسلة نصية:

```
print("".join(reversed(balloon)))
.noollab a sah ymmaS
```

لم نرغب في إضافة أيّة سلسلة نصية إلى أخرى، لذا أبقينا على السلسلة النصية فارغة دون

محتوى داخلها. الدالة `str.join()` مفيدة أيضًا لجمع قائمة (list) من السلاسل النصية وإخراجها إلى سلسلة وحيدة. لُنشئ سلسلة نصية يُفصل بين كلماتها بفاصلة من القائمة الآتية:

```
print(",".join(["sharks", "crustaceans", "plankton"]))
sharks,crustaceans,plankton
```

إذا أردتَ وضع فاصلة ثم فراغ بين القيم في المثال السابق، فيمكنك أن تُعيد كتابة التعليمة البرمجية السابقة لإضافة فراغ بعد الفاصلة كما يلي:

```
", ".join(["sharks", "crustaceans", "plankton"])
```

وكما نستطيع جمع السلاسل النصية مع بعضها بعضًا، نستطيع أيضًا تجزئتها، وذلك عبر

الدالة `str.split()`:

```
print(balloon.split())
['Sammy', 'has', 'a', 'balloon.']
```

ستُعيد الدالة `str.split()` قائمة (list) تحوي سلاسل نصية كانت مفصولةً بالفراغات في

السلسلة النصية الأصلية إذا لم يُمرَّر معاملٌ لتحديد محرف الفصل. يمكنك أيضًا استخدام الدالة

`str.split()` لحذف أجزاء معيّنة من السلسلة النصية الأصلية، فلنحاول مثلًا حذف الحرف `a`:

```
print(balloon.split("a"))
['S', 'mmy h', 's ', ' b', 'lloon.']
```

حُذِفَ الحرف `a` من السلسلة النصية وأصبح الناتج مقسومًا عند كل ورود للحرف `a` مع

الإبقاء على الفراغات. تأخذ الدالة `str.replace()` سلسلة نصية وتُعيد نسخةً محدّثةً منها بعد

إجراء بعض عمليات الاستبدال عليها. لنفترض أنَّ البالون الذي يملكه سامي قد ضاع، ولعدم

امتلاك سامي للبالون في الوقت الراهن، فستُبدل الكلمة `"has"` إلى `"had"`:


```
print(balloon.replace("has", "had"))
```

أول سلسلة نصية داخل أقواس الدالة `replace()` هي السلسلة النصية التي نريد استبدالها، والسلسلة النصية الثانية هي السلسلة التي نريد وضعها بدلاً من الأولى. ناتج تنفيذ السطر السابق هو:

```
Sammy had a balloon.
```

استخدام دوال تعديل السلاسل النصية مثل `str.join()` و `str.split()` و `str.replace()` سيمنحك تحكمًا كبيرًا بمعالجة السلاسل النصية في بايثون.

8. دوال الإحصاء

بعد أن تعرفنا على آلية فهرسة المحارف في السلاسل النصية، حان الوقت لمعاينة بعض الدوال التي تُحصي السلاسل النصية أو تعيد أرقام الفهارس. يمكننا أن نستفيد من ذلك بتحديد عدد المحارف التي نريد استقبالها من مدخلات المستخدم، أو لموازنة السلاسل النصية. لدى السلاسل النصية -كغيرها من أنواع البيانات- عدّة دوال تُستخدم للإحصاء. لننظر أولاً إلى الدالة `len()` التي تُعيد طول أيّ نوع متسلسل من البيانات، بما في ذلك الأنواع `string` و `list` و `tuple` و `dictionary`. لنطبع طول السلسلة النصية `ss`:

```
print(len(ss))
# 12
```

طول السلسلة النصية `"Sammy Shark!"` هو 12 محرفًا، بما في ذلك الفراغ وعلامة التعجب. بدلاً من استخدام متغيّر، فلنحاول مباشرةً تمرير سلسلة نصية إلى الدالة `len()`:

```
print(len("Let's print the length of this string. "))
# 38
```

الدالة `len()` تُحصى العدد الإجمالي من المحارف في سلسلة نصية. إذا أردنا إحصاء عدد مرات تكرار محرف أو مجموعة من المحارف في سلسلة نصية، فيمكننا استخدام الدالة `str.count()`، لنحاول إحصاء الحرف `a` في السلسلة النصية `ss`:

```
print(ss.count("a"))
# 2
```

يمكننا البحث عن محرفٍ آخر:

```
print(ss.count("s"))
# 0
```

صحيح أنَّ الحرف `s` قد ورد في السلسلة النصية، إلا أنَّه من الضروري أن تبقي بذهنك أنَّ بايثون حساسةٌ لحالة الأحرف، فلو أردنا البحث عن حروفٍ معيَّنة بغض النظر عن حالتها، فعلينا حينها استخدام الدالة `str.lower()` لتحويل حروف السلسلة النصية إلى حروفٍ صغيرة. لنحاول استخدام الدالة `str.count()` مع سلسلة من المحارف:

```
likes = "Sammy likes to swim in the ocean, likes to spin up servers, and likes to smile."
print(likes.count("likes"))
```

الناتج هو 3، إذ تتواجد مجموعة المحارف "likes" ثلاث مرات في السلسلة النصية الأصلية. يمكننا أيضًا معرفة موقع الحرف أو مجموعة الحروف في السلسلة النصية، وذلك عبر الدالة `str.find()`، وسيُعاد موضع المحرف بناءً على رقم فهرسه. يمكننا أن نعرف متى يقع أوَّل حرف `m` في السلسلة النصية `ss` كالآتي:

```
print(ss.find("m"))
# 2
```

أول مرة يقع فيها الحرف m في الفهرس 2 من السلسلة "Sammy Shark"، يمكنك أن تراجع بداية هذا الدرس لرؤية جدول يبيّن ارتباطات المحارف مع فهارسها في السلسلة السابقة. لنرى الآن مكان أول ظهور لمجموعة المحارف likes في السلسلة "likes":

```
print(likes.find("likes"))
# 6
```

أول مرة تظهر فيها السلسلة "likes" هي في الفهرس 6، أي مكان وجود الحرف l من likes. ماذا لو أردنا أن نعرف موضع ثاني تكرار للكلمة likes؟ نستطيع فعل ذلك بتمرير معاملٍ ثانٍ إلى الدالة str.find() الذي سيجعلها تبدأ بحثها من ذاك الفهرس، فبدلاً من البحث من أول السلسلة النصية سنبحث انطلاقاً من الفهرس 9:

```
print(likes.find("likes", 9))
# 34
```

بدأ البحث في هذا المثال من الفهرس 9، وكانت أول مطابقة للسلسلة النصية "likes" عند الفهرس 34. إضافةً إلى ذلك، يمكننا تحديد نهاية إلى مجال البحث بتمرير معامل ثالث. وكما عند تقسيم السلاسل النصية، يمكننا استخدام أرقام الفهارس السالبة للعد عكسيًا:

```
print(likes.find("likes", 40, -6))
# 64
```

يبحث آخر مثال عن موضع السلسلة النصية "likes" بين الفهرس 40 و -6، ولمّا كان المعامل الأخير هو رقم سالب، فسيبدأ العد من نهاية السلسلة الأصلية.

دوال الإحصاء مثل `len()` و `str.count()` و `str.find()` مفيدة في تحديد طول

السلسلة النصية وعدد حروفها وفهارس ورود محارف معيّنة فيها.

9. خلاصة الفصل

لقد تعلمنا في هذا الفصل أساسيات التعامل مع السلاسل النصية في لغة بايثون 3. بما في

ذلك إنشاءها وطباعتها وجمعها وتكرارها، إضافةً إلى تخزينها في متغيرات، وهذه هي المعلومات

الأساسية التي عليك فهمها للانطلاق في تعاملك مع السلاسل النصية في برامج بايثون 3.